Numerical Analysis: Midterm

(**40 marks**)

Solutions

Urbain Vaes

Fall 2024

Question 1 (Floating point arithmetic, **10 marks**)**.** True or false?

1. Let $(\bullet)_2$ denote binary representation. It holds that $(0.1011)_2 + (0.0101)_2 = 1$.
2. Let $(\bullet)_3$ denote base 3 representation. It holds that $(1000)_3 \times (0.002)_3 = 2$.
3. A natural number with binary representation $(b_4 b_3 b_2 b_1 b_0)_2$ is even if and only if $b_0 = 0$.
4. In Julia, `Float64(.4) == Float32(.4)` evaluates to `true`.
5. Machine addition $\widehat{+}$ is a commutative operation. More precisely, given any two double-precision floating point numbers $x \in \mathbf{F}_{64}$ and $y \in \mathbf{F}_{64}$, it holds that $x \mathbin{\widehat{+}} y = y \mathbin{\widehat{+}} x$.
6. Let $\mathbf{F}_{32}$ and $\mathbf{F}_{64}$ denote respectively the sets of single and double precision floating point numbers. It holds that $\mathbf{F}_{32} \subset \mathbf{F}_{64}$.
7. In Julia, `eps(Float16)` returns the smallest strictly positive number that can be represented exactly in the `Float16` format.
8. Let $\mathbf{F}_{64}$ be the set of double precision floating point numbers. For any $x \in \mathbf{R}$ such that $x \in \mathbf{F}_{64}$, it holds that $x + 1 \in \mathbf{F}_{64}$.
9. Let $x \in \mathbf{R}$ and $y \in \mathbf{R}$ be two numbers that are exactly representable in the `Float64` format. Then $x \mathbin{\widehat{+}} y = x + y$: machine addition is exact in this case.
10. It holds that $(0.\overline{2200})_3 = (0.9)_{10}$.

Solution.

1. **True**: $(0.1011)_2 + (0.0101)_2 = \frac{11}{16} + \frac{5}{16} = 1$.
2. **True**: $(1000)_3 = 27$ and $(0.002)_3 = \frac{2}{27}$.
3. **True**: the number equals $2(8b_4 + 4b_3 + 2b_2 + b_1) + b_0$, which is even if and only if $b_0 = 0$.
4. **False**: $0.4 = (0.\overline{0110})_2$ is not exactly representable, and its roundings to single and double precision differ.
5. **True**: $x \mathbin{\widehat{+}} y$ is the rounding of $x + y = y + x$.
6. **True**: any single precision number, including subnormal numbers, has at most 24 signifi-

cant bits and an exponent within the range of the double precision format, so it is exactly representable in double precision.

7. **False**: `eps(Float16)` $= 2^{-10}$ is the distance between 1 and the next `Float16` number. The smallest strictly positive `Float16` number is the subnormal number $2^{-24} \approx 6 \times 10^{-8}$.
8. **False**: for example, $x = 2^{53} + 2 \in \mathbf{F}_{64}$, but $x + 1 = 2^{53} + 3 \notin \mathbf{F}_{64}$. (Another example: $x = 2^{-60}$.)
9. **False**: 1 and 2^{-60} are representable, but $1 + 2^{-60}$ is not, and $1 \mathbin{\widehat{+}} 2^{-60} = 1$.
10. **True**: if $x = (0.\overline{2200})_3$, then $3^4 x = (2200)_3 + x = 72 + x$, so $x = \frac{72}{80} = 0.9$.

△

Question 2 (Interpolation and approximation, **10 marks**)**.** Are the following assertions true or false? Throughout this exercise, we use the notation $x_i^n = i/n$. The notation $\mathbf{P}(n)$ denotes the set of polynomials of degree less than or equal to n. We proved in class that, for any function $u\colon \mathbf{R} \to \mathbf{R}$ and for all $n \in \mathbf{N}_{>0}$, there exists a unique polynomial $p_n \in \mathbf{P}(n)$ such that

$$\forall i \in \{0, \dots, n\}, \qquad p_n(x_i^n) = u(x_i^n). \tag{1}$$

1. If u is not the zero function, then the degree of p_n is exactly n.
2. If $u(x) = 2x + 1$, then $p_n = u$ for all $n \in \{1, 2, 3, \dots\}$.
3. Fix $u(x) = 1 + \sin(57\pi x)$. Then $p_3(x) = 1$.
4. Fix $u(x) = (2x - 1)^3$. Then $p_2(x) = 2x - 1$.
5. Fix $n \in \mathbf{N}_{>0}$ and suppose that $u\colon \mathbf{R} \to \mathbf{R}$ is a smooth function. There exists a constant $K > 0$ independent of x such that

$$\forall x \in \mathbf{R}, \qquad u(x) - p_n(x) = K \prod_{i=0}^{n} (x - x_i^n)\,.$$

6. It holds that

$$\forall x \in [0, 1], \qquad \left|(x - x_0^n) \dots (x - x_n^n)\right| \leqslant n! \left(\frac{1}{n}\right)^{n+1}.$$

7. In Julia, assuming that n and the function u have already been defined, the following code enables to calculate the interpolating polynomial p_n of u:

```
using Polynomials
# Assume `n=5` and `u` have already been defined
x = LinRange(0, 1, n + 1)
p = fit(x, u)
```

8. Let Δ denote the finite difference operator: for a function $f\colon \mathbf{R} \to \mathbf{R}$, the function $\Delta f\colon \mathbf{R} \to \mathbf{R}$ is defined as

$$\Delta f(x) = f(x + 1) - f(x).$$

Then $f \in \mathbf{P}(n)$ if and only if $\Delta^{n+1} f = 0$. Here Δ^{n+1} denotes the composition of $n+1$ applications of the operator Δ.

9. In Julia, the following code enables to fit the data `x` and `y` by a straight line.

```
using Polynomials
x = [1, 2, 3, 4]
y = [16, 9, 4, 1]
p = fit(x, y, 1)
```

10. Using Chebyshev instead of equidistant points can improve on the quality of the interpolation.

Solution. **1. False**: for example, if $u(x) = x$ and $n = 2$, then $p_2 = u$ has degree 1.

2. True: $u \in \mathbf{P}(1) \subset \mathbf{P}(n)$ satisfies the interpolation conditions, so $p_n = u$ by uniqueness.

3. True: $\sin(57\pi x_i^3) = \sin(19\pi i) = 0$ for $i \in \{0, 1, 2, 3\}$, so $u(x_i^3) = 1$ and the constant polynomial 1 interpolates the data.

4. True: $u(0) = -1$, $u(\frac{1}{2}) = 0$ and $u(1) = 1$, and the polynomial $2x - 1 \in \mathbf{P}(2)$ takes these values at these points.

5. False: the interpolation error is $u(x) - p_n(x) = \frac{u^{(n+1)}(\xi(x))}{(n+1)!} \prod_{i=0}^{n} (x - x_i^n)$, where $\xi(x)$ depends on x in general. For example, with $n = 1$ and $u(x) = x^3$, $u(x) - p_1(x) = x^3 - x = (x+1)x(x-1)$ is not of the form $Kx(x-1)$.

6. True: let $h = \frac{1}{n}$ and $x \in [x_j^n, x_{j+1}^n]$. Then $|(x - x_j^n)(x - x_{j+1}^n)| \leqslant \frac{h^2}{4}$, and $|x - x_i^n| \leqslant (j+1-i)h$ for $i < j$ and $|x - x_i^n| \leqslant (i-j)h$ for $i > j+1$. Therefore

$$\left|(x - x_0^n) \dots (x - x_n^n)\right| \leqslant \frac{h^2}{4} (j+1)!\,(n-j)!\,h^{n-1} \leqslant \frac{n!}{4} h^{n+1},$$

where we used that $(j+1)!\,(n-j)! = \frac{(n+1)!}{\binom{n+1}{j+1}} \leqslant \frac{(n+1)!}{n+1} = n!$ for $0 \leqslant j \leqslant n-1$.

7. False: the function `fit` expects the values of u at the nodes, not the function itself: the last line should be `p = fit(x, u.(x))`.

8. False, strictly speaking. If $f \in \mathbf{P}(n)$, then $\Delta f \in \mathbf{P}(n-1)$ (the leading terms cancel), and so $\Delta^{n+1} f = 0$. However, the converse does not hold for a general function $f \colon \mathbf{R} \to \mathbf{R}$: for example, $f(x) = \sin(2\pi x)$ satisfies $\Delta f = 0$, but $f \notin \mathbf{P}(0)$. (The converse holds if f is known to be a polynomial.)

9. True: `fit(x, y, 1)` returns the polynomial of degree 1 minimizing the sum of squared errors, here $p(x) = 20 - 5x$.

10. True: Chebyshev nodes minimize $\max_x \left|\prod_{i=0}^{n} (x - x_i)\right|$, which appears in the interpolation error formula, and they avoid the Runge phenomenon.

△

Question 3 (Numerical integration, **10 marks**)**.** The Gauss–Legendre quadrature formula with n nodes is an approximate integration formula of the form

$$I(u) := \int_{-1}^{1} u(x)\,\mathrm{d}x \approx \sum_{i=1}^{n} w_i\, u(x_i) =: \widehat{I}_n(u), \tag{2}$$

which is exact when u is a polynomial of degree less than or equal to $2n-1$. (Note that the nodes are here numbered starting from 1.)

1. (**5 marks**) Find the nodes and weights of the Gauss–Legendre rule with $n = 3$ nodes, without using any formula (unless you prove it beforehand).

   Hint: Recall that a necessary and sufficient condition in order for (2) to be satisfied for any polynomial $p \in \mathbf{P}(5)$ is that

   $$\int_{-1}^{1} x^d \,\mathrm{d}x = \sum_{i=1}^{n} w_i x_i^d, \qquad \text{for all } d \in \{0,1,2,3,4,5\}.$$

   Furthermore, given the symmetry of the problem, it is reasonable to look for a solution of the following form, which enables to reduce the number of unknowns.

   $$(x_1, x_2, x_3, w_1, w_2, w_3) = (-x, 0, x, w_1, w_2, w_1).$$

2. (**5 marks**) Are the following assertions true of false :

   - The degree of precision of the composite trapezium rule is 2.
   - The composite Simpson rule can be used to integrate exactly a quadratic polynomial.
   - The degree of precision of the following rule is 1:

     ```
     function my_integrate(f, a, b)
         x = LinRange(a, b, 100)
         h = x[2] - x[1]
         return h * sum(f, x[1:end-1])
     end
     ```

   - The degree of precision of the following integration rule is 2:

     $$\int_{-1}^{1} f(x) \,\mathrm{d}x \approx 2f(0) + \frac{1}{3} f''(0).$$

   - Suppose that $u \colon \mathbf{R} \to \mathbf{R}$ is a smooth function, and let $\widehat{I}_n(u)$ denote an approximation of the integral $I(u) := \int_{-1}^{1} u(x) \,\mathrm{d}x$ by the composite trapezium approximation with n points. Let

     $$\widehat{J}_n(u) = 2\widehat{I}_{2n}(u) - \widehat{I}_n(u).$$

     It holds that

     $$\lim_{n \to \infty} n^2 \left| I(u) - \widehat{J}_n(u) \right| = 0.$$

Solution. **1.** With the suggested form, the conditions for odd d are satisfied automatically by symmetry. The conditions for $d \in \{0, 2, 4\}$ read

$$2w_1 + w_2 = 2, \qquad 2w_1 x^2 = \frac{2}{3}, \qquad 2w_1 x^4 = \frac{2}{5}.$$

Dividing the third equation by the second gives $x^2 = \frac{3}{5}$, then $w_1 = \frac{1}{3x^2} = \frac{5}{9}$ and $w_2 = 2 - 2w_1 = \frac{8}{9}$. Therefore

$$(x_1, x_2, x_3) = \left(-\sqrt{\frac{3}{5}}, 0, \sqrt{\frac{3}{5}}\right), \qquad (w_1, w_2, w_3) = \left(\frac{5}{9}, \frac{8}{9}, \frac{5}{9}\right).$$

2.

- **False**: the composite trapezium rule integrates exactly the piecewise linear interpolant, so it is exact for polynomials of degree 1 but not for x^2. Its degree of precision is 1.
- **True**: Simpson's rule has degree of precision 3, so the composite rule is exact for polynomials of degree up to 3, in particular quadratic polynomials.
- **False**: this is the composite left rectangle rule, which is exact for constant functions but not for $u(x) = x$. For example, with $a = 0$ and $b = 1$ it gives $\frac{49}{99}$ instead of $\frac{1}{2}$. Its degree of precision is 0.
- **False**: the rule is exact for 1, x^2 (since $\frac{1}{3} \times 2 = \frac{2}{3}$), and for x and x^3 (by symmetry), but not for x^4 (it gives 0 instead of $\frac{2}{5}$). Its degree of precision is 3, not 2.
- **False**: the error of the composite trapezium rule satisfies $I(u) - \widehat{I}_n(u) \approx \frac{C}{n^2}$ for large n, for some constant C, and so

$$I(u) - \widehat{J}_n(u) = 2\big(I(u) - \widehat{I}_{2n}(u)\big) - \big(I(u) - \widehat{I}_n(u)\big) \approx \frac{2C}{4n^2} - \frac{C}{n^2} = -\frac{C}{2n^2},$$

so $n^2|I(u) - \widehat{J}_n(u)| \to \frac{|C|}{2}$, which is nonzero in general. The correct Richardson extrapolation is $\frac{4\widehat{I}_{2n}(u) - \widehat{I}_n(u)}{3}$.

△

💻 **Computer exercise 1** (Interpolation, **10 marks**)**.** Consider the following data:

Time (hours)	**Temperature (°C)**
6	10.5
9	15.0
12	20.2
15	25.1
18	22.8
21	17.4

Table 1: Recorded temperatures at different times of the day.

We wish to approximate the temperature as a smooth function of time. To this end, calculate the interpolating polynomial p_{int}, as well as the best quadratic polynomial approximation p_{app} (in the sense that the sum of square errors is minimized). You may use the `Polynomials` library. Plot on the same graph:

- The data points using `scatter`;
- The polynomial p_{int} interpolating the data points;
- The quadratic polynomial p_{app} that best approximates the data, in the sense of least squares.

Solution. The interpolating polynomial has degree at most 5 (six data points), and `fit(t, T, 2)` computes the least squares approximation of degree 2:

```
using Polynomials, Plots
t = [6, 9, 12, 15, 18, 21]
T = [10.5, 15.0, 20.2, 25.1, 22.8, 17.4]
p_int = fit(t, T)     # degree 5, interpolation
p_app = fit(t, T, 2)  # degree 2, least squares
scatter(t, T, label="Data", xlabel="Time (hours)", ylabel="Temperature (C)")
plot!(p_int, 6, 21, label="Interpolation")
plot!(p_app, 6, 21, label="Least squares (degree 2)")
```

We obtain

$$p_{\text{int}}(t) \approx -123.3 + 61.10t - 10.81t^2 + 0.9298t^3 - 0.03781t^4 + 0.0005796t^5,$$
$$p_{\text{app}}(t) \approx -14.18 + 4.857t - 0.1577t^2.$$

The interpolating polynomial passes through all the data points, while the quadratic approximation captures the overall trend: the temperature increases in the morning, reaches a maximum in the afternoon and then decreases. △

💻 **Computer exercise 2** (Numerical integration, **10 marks**)**.** Boole's integration rule reads

$$\int_{-1}^{1} u(x)\,\mathrm{d}x \approx \frac{7}{45}u(-1) + \frac{32}{45}u\left(-\frac{1}{2}\right) + \frac{12}{45}u\left(0\right) + \frac{32}{45}u\left(\frac{1}{2}\right) + \frac{7}{45}u(1).$$

- Write a function `comp_boole(u, a, b, N)`, which returns an approximation of the integral

$$I(u) = \int_a^b u(x)\,\mathrm{d}x$$

obtained by partitioning the integration interval $[a, b]$ into N cells, and applying Boole's rule within each cell.

- Take $u(x) = \cos(x)$, $a = -1$ and $b = 1$. Plot the evolution of the error for $N \in \{1, \ldots, 200\}$.

- Estimate the order of convergence with respect to N, i.e. find α such that

$$|\widehat{I}_N - I| \propto CN^{-\alpha},$$

where I denotes the exact value of the integral and $\widehat{I}_N$ denotes its approximation by the composite Boole's rule. Use the method you prefer in order to find α. You can, for example, use the function `fit` from the `Polynomials` package to find a linear approximation of the form

$$\log|\widehat{I}_N - I| \approx \log(C) - \alpha\log(N).$$

Solution.

- Boole's rule on $[a, b]$ is obtained by the change of variable $x = \frac{a+b}{2} + \frac{b-a}{2}t$:

```
function boole(u, a, b)
    h = (b - a) / 4
    return (b - a) / 90 * (7u(a) + 32u(a + h) + 12u(a + 2h) + 32u(a + 3h) + 7u(b))
end

function comp_boole(u, a, b, N)
    x = range(a, b, length = N + 1)
    return sum(boole(u, x[i], x[i+1]) for i in 1:N)
end
```

- The exact integral is $I = 2\sin(1)$:

```
using Plots
Ns = 1:200
errors = [abs(comp_boole(cos, -1, 1, N) - 2sin(1)) for N in Ns]
plot(Ns, errors, xaxis=:log, yaxis=:log, label="Error", xlabel="N")
```

  The error decreases from about 6×10^{-5} for $N = 1$ to about 2×10^{-11} for $N = 12$, and then reaches the level of round-off errors ($\approx 10^{-16}$) for large N.

- We fit a straight line in log-log scale, using only values of N for which the error is much larger than round-off errors:

```
using Polynomials
Ns = 1:20
p = fit(log.(Ns), log.(errors[Ns]), 1)
alpha = -p[1]  # approximately 6
```

  We obtain $\alpha \approx 6$. This is consistent with theory: Boole's rule has degree of precision 5, so the error in each cell of size $h = \frac{b-a}{N}$ is $\mathcal{O}(h^7)$, and the total error is $\mathcal{O}(Nh^7) = \mathcal{O}(N^{-6})$.

△